

ALL 8 ARB ISSUES RESOLVED ✓

YUNO AI AGENT PLATFORM

End-to-End Architecture & System Documentation

Use Case • Data Flow • Activity • Deployment
Auth Flow • CI/CD • Availability • Resiliency
DR/BCP • Backup • NFR Matrix • ARB Remediation

Table of Contents

1. Project Overview & Requirements	3
2. Use Case Diagram	4
3. Data Flow Diagram	5
4. Authentication Flow (ARB-001)	6
5. Activity Diagram — Auth + Execution	7
6. Deployment Diagram	8
7. CI/CD Pipeline Flow (ARB-007)	9
8. Availability Architecture	10
9. Resiliency Architecture	11
10. Disaster Recovery & BCP	12
11. Backup & Recovery	13
12. NFR Matrix (26 controls)	14
13. ARB Remediation — All 8 Resolved	15
14. ADRs (8 records)	16
15. Technology Reference Stack	17

1. Project Overview & Requirements

The **Yuno AI Agent Orchestration Platform v3.0** is a production-ready, full-stack multi-agent AI system. Version 3.0 resolves all 8 ARB open issues and is deployment-ready with PostgreSQL, Redis, JWT auth, rate limiting, distributed tracing, and a full CI/CD pipeline.

ID	Feature	Description
FR-01	Agent Management	Create agents: personality, tools, memory, guardrails, schedules
FR-02	Workflow Builder	React Flow visual canvas — drag-drop, conditional edges
FR-03	LangGraph Runtime	StateGraph execution, tool calling, iteration guard
FR-04	Telegram	Long-poll bot — receive and reply to messages
FR-05	Live Monitoring	WebSocket real-time events, token + cost tracking per run
FR-06	Templates	Research Report and Customer Support starter templates
FR-07	REST API	Full CRUD — agents, workflows, runs; OpenAPI at /docs
FR-08	Tools	Web search, calculator, HTTP, time, text summariser
FR-09	Auth (ARB-001)	JWT Bearer + API Key — all routes protected
FR-10	Rate Limit (ARB-005)	100 req/min global, 10 runs/min workflow
FR-11	Tracing (ARB-006)	OpenTelemetry → Jaeger auto-instrumented
FR-12	CI/CD (ARB-007)	GitHub Actions: lint+test+scan → Docker → deploy

2. Use Case Diagram

All actors (Admin, Developer, End User, Telegram User, OpenAI API, GitHub Actions) and 16 use cases within the system boundary. New in v3.0: JWT Authenticate, Rate-Limited Exec use cases.

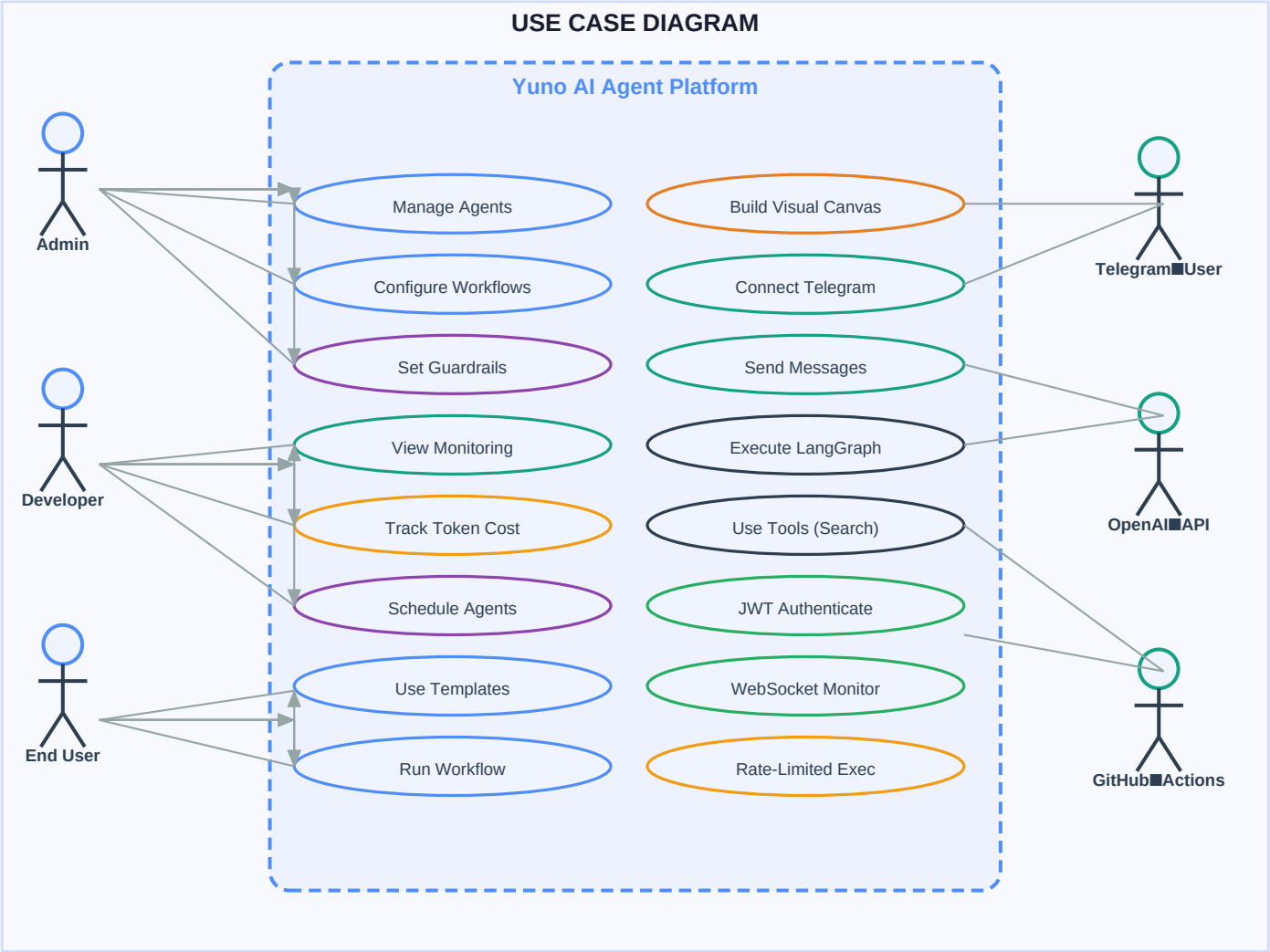


Figure 2: Use Case Diagram

3. Data Flow Diagram (Level 1)

P1 JWT Auth Middleware is the first processing stage — all API flows pass through auth. P7 Redis EventBus replaces the in-process queue. 4 data stores persisted to PostgreSQL.

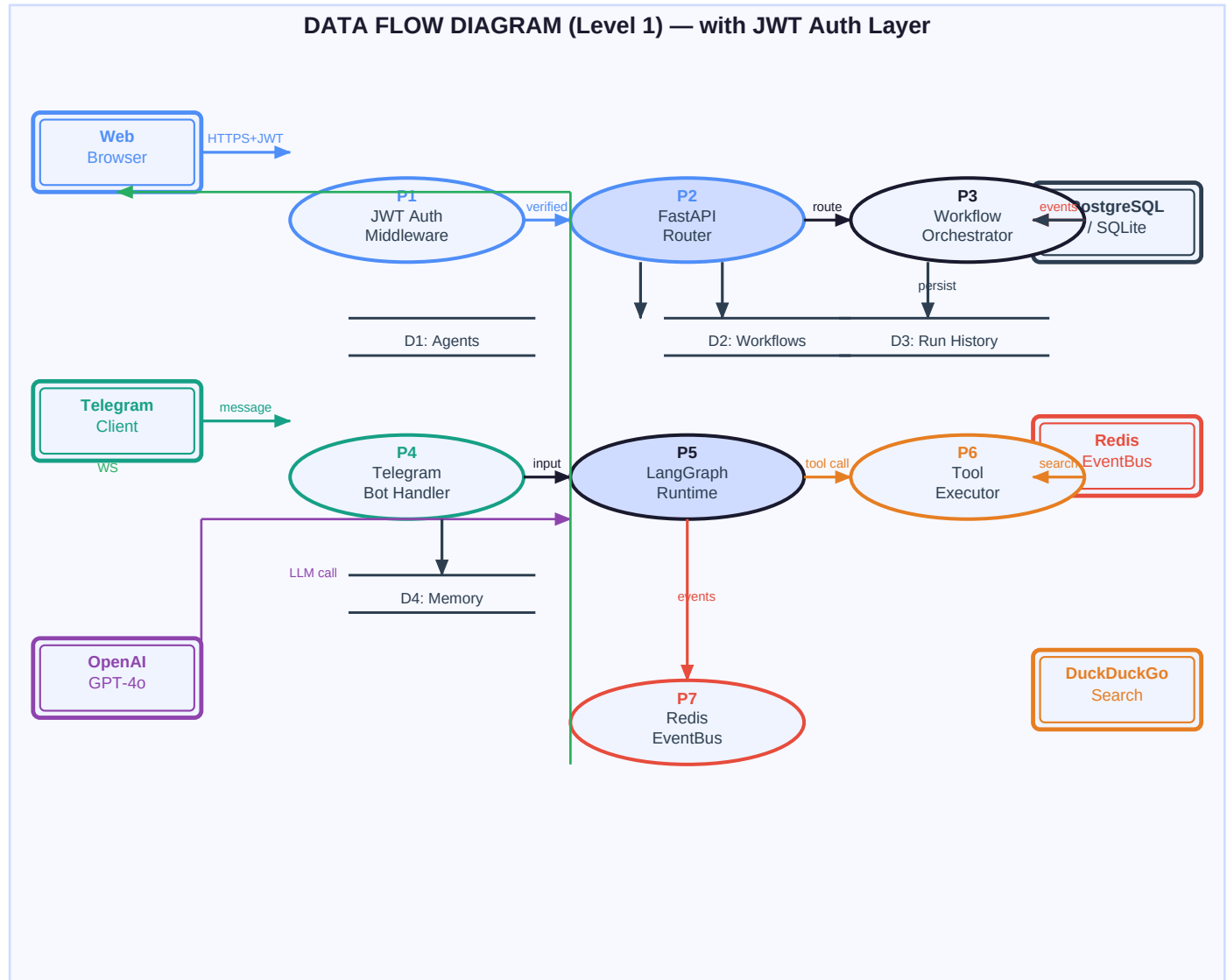


Figure 3: Data Flow Diagram (Level 1)

4. Authentication Flow (ARB-001)

Three auth paths: (A) Login via POST /auth/token — bcrypt verify → HS256 JWT returned. (B) Protected route with Bearer token — jwt.decode() validates exp+sub. (C) X-API-Key header for service accounts. WebSocket auth via ?token= query param.

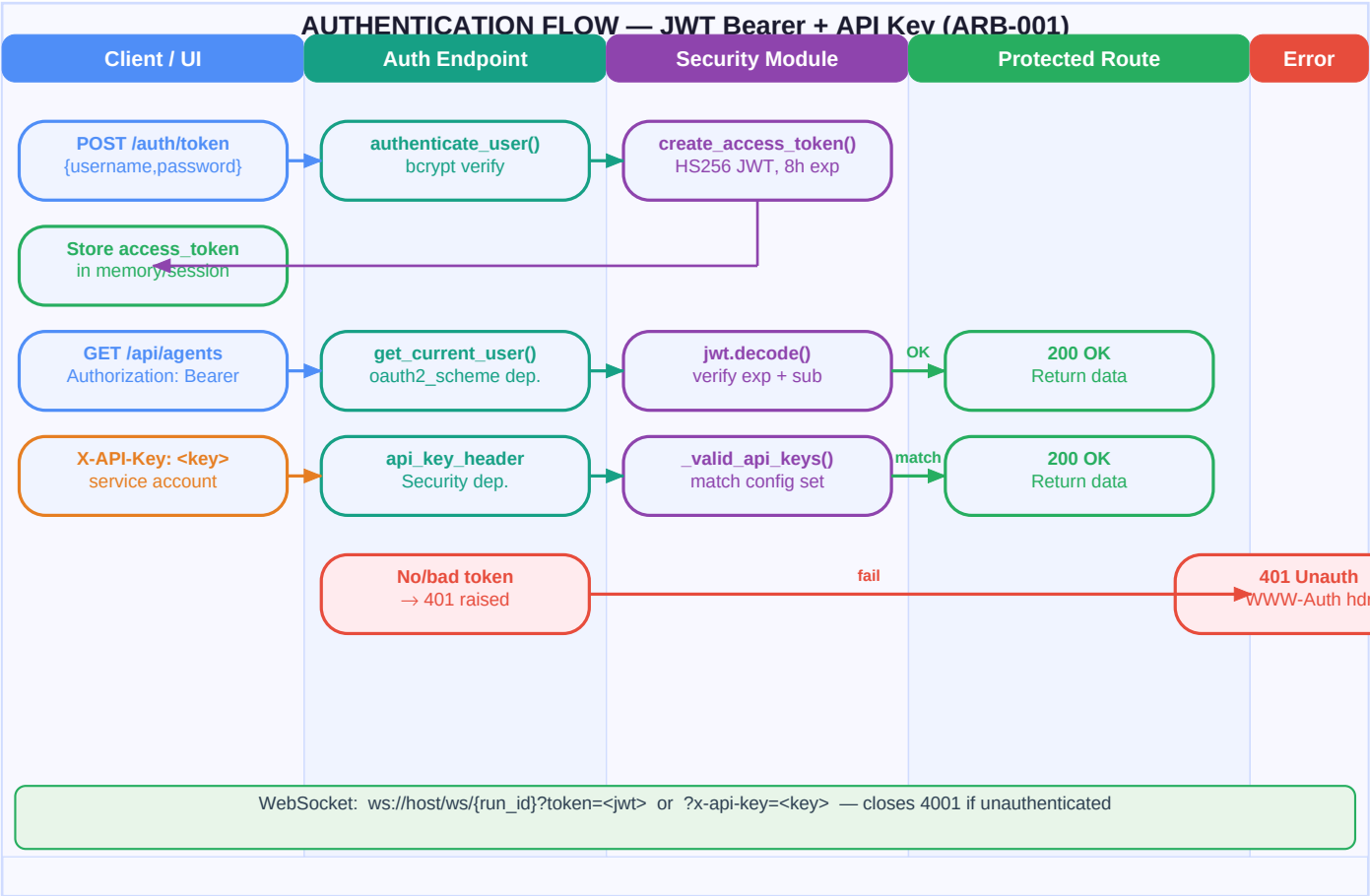


Figure 4: Authentication Flow (ARB-001)

5. Activity Diagram — Auth + Execution

Five-swimlane flow: Client → Auth Layer (JWT/API key check + 401 path) → FastAPI (rate limit check + 429 path) → LangGraph (execute, tool calls, edge routing) → Tools (search/HTTP). Events published to Redis after each agent step.

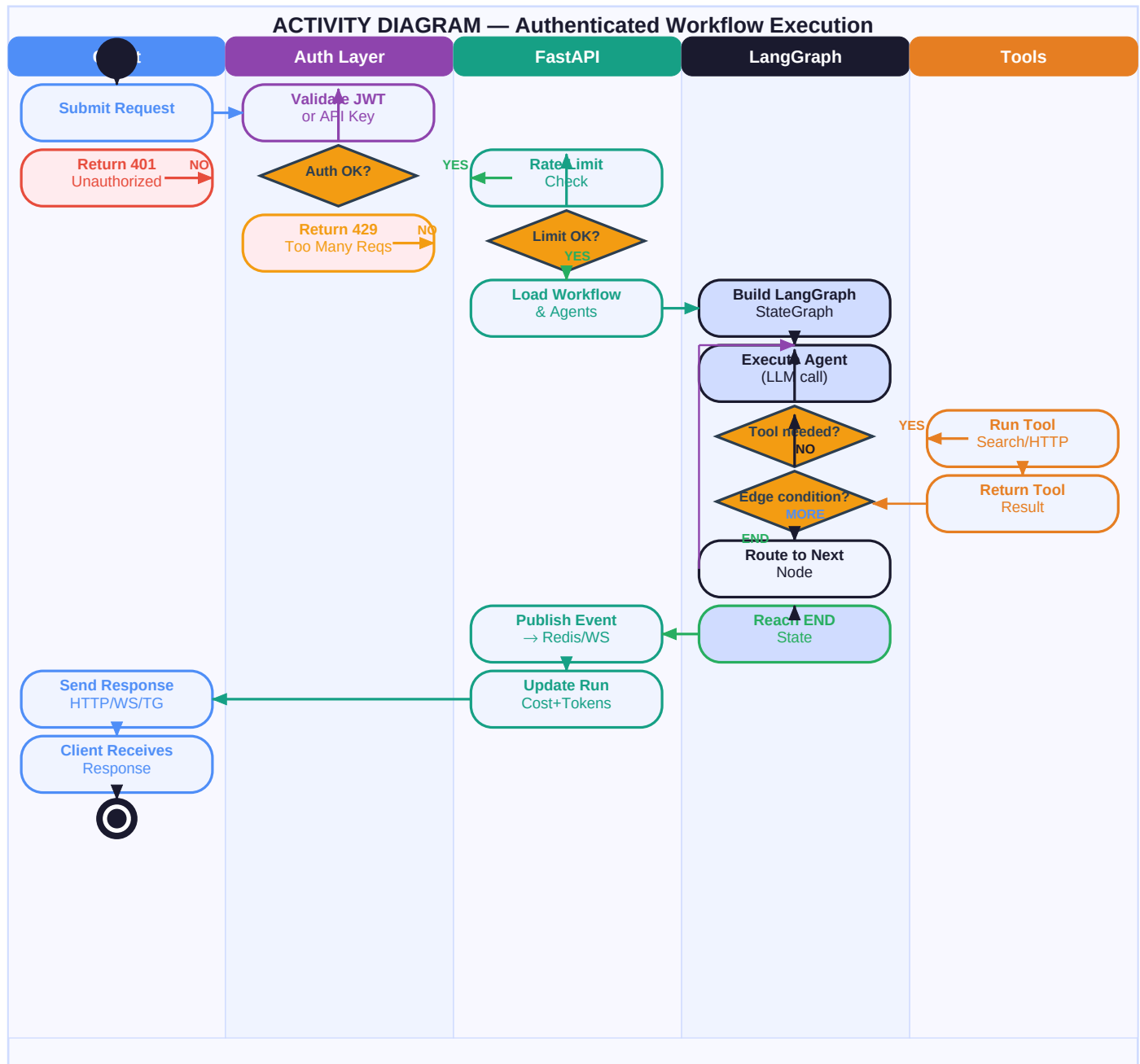


Figure 5: Activity Diagram — Auth + Execution

6. Deployment Diagram

Production Docker Compose stack: nginx (SSL + rate limit layer), frontend, backend, PostgreSQL 16, Redis 7, Telegram bot, Jaeger all-in-one. GitHub Actions CI/CD deploys via SSH on merge to main. Secrets in GitHub Actions encrypted secrets.

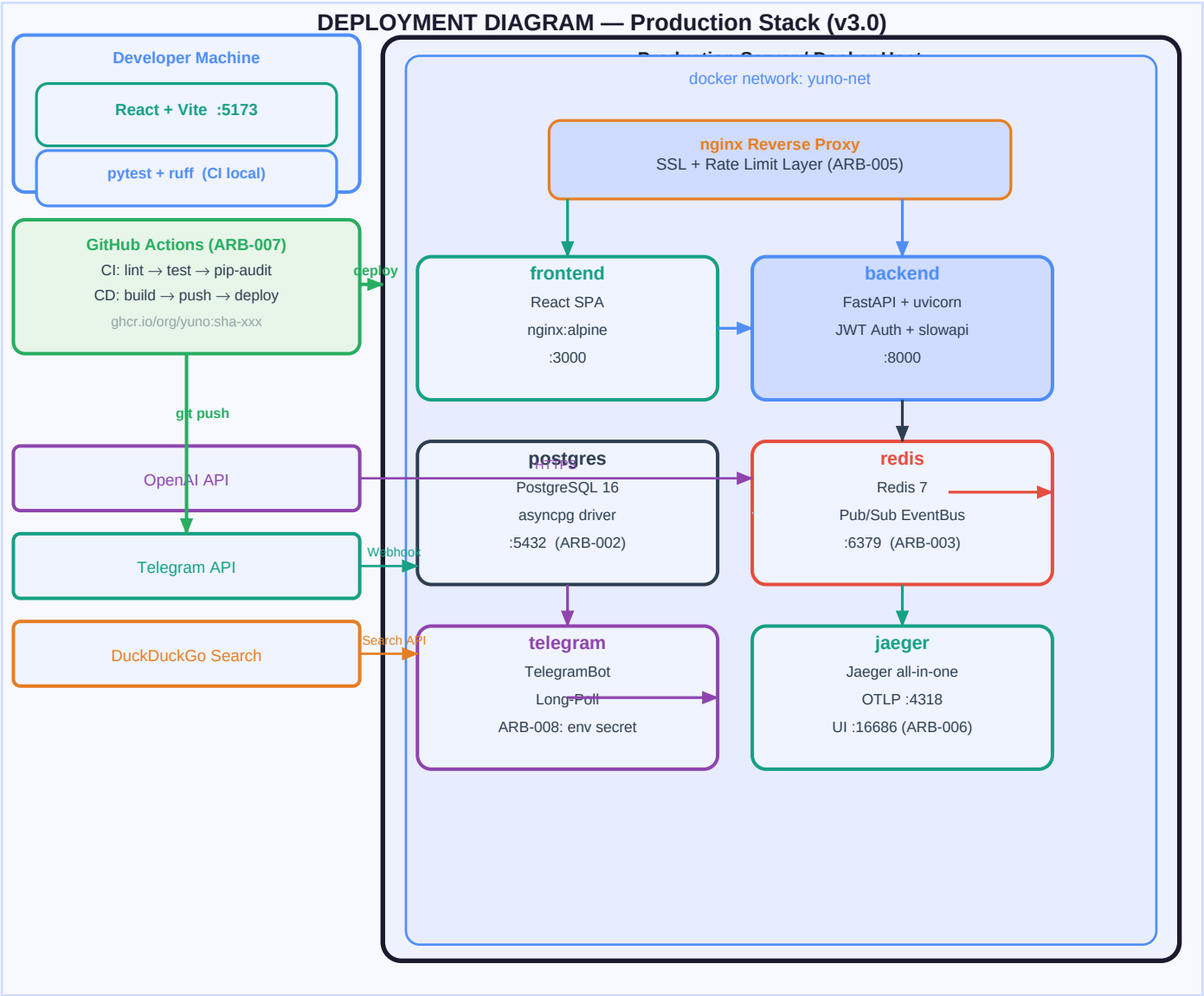


Figure 6: Deployment Diagram

7. CI/CD Pipeline Flow (ARB-007)

CI: checkout → pip install → ruff lint → mypy → pytest (with Redis service) → pip-audit. Gate: all checks must pass. CD: build backend + frontend Docker images → push to ghcr.io → SSH deploy → smoke test GET /health + JWT auth check.

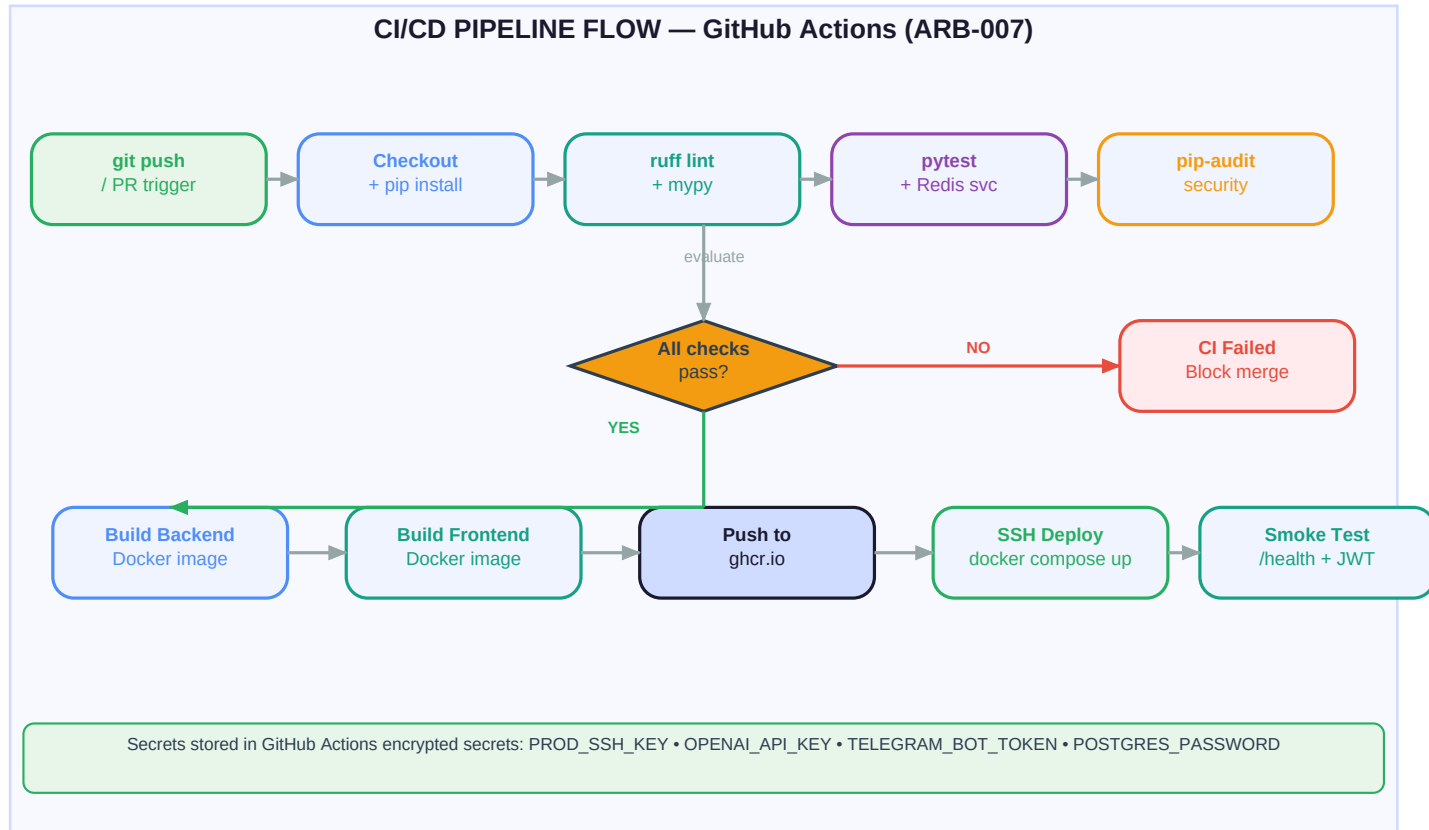


Figure 7: CI/CD Pipeline Flow (ARB-007)

8. Availability Architecture

nginx load balancer → 3 API instances (primary, replica, standby). PostgreSQL primary+replica with streaming replication. Redis 7 cluster for EventBus and rate-limit storage. OpenTelemetry → Jaeger + Grafana. SLA 99.5%, RTO 4h, RPO 1h.

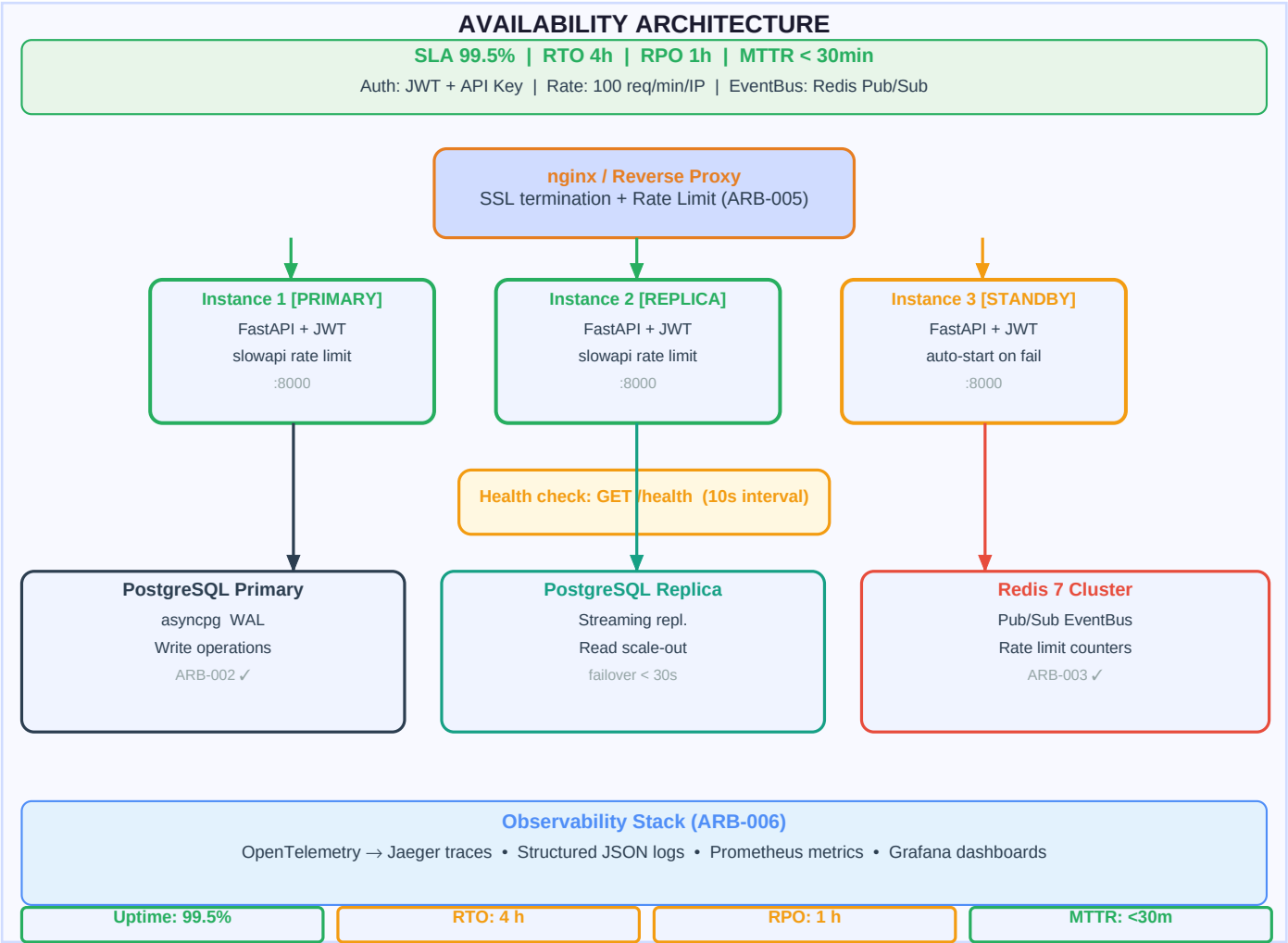
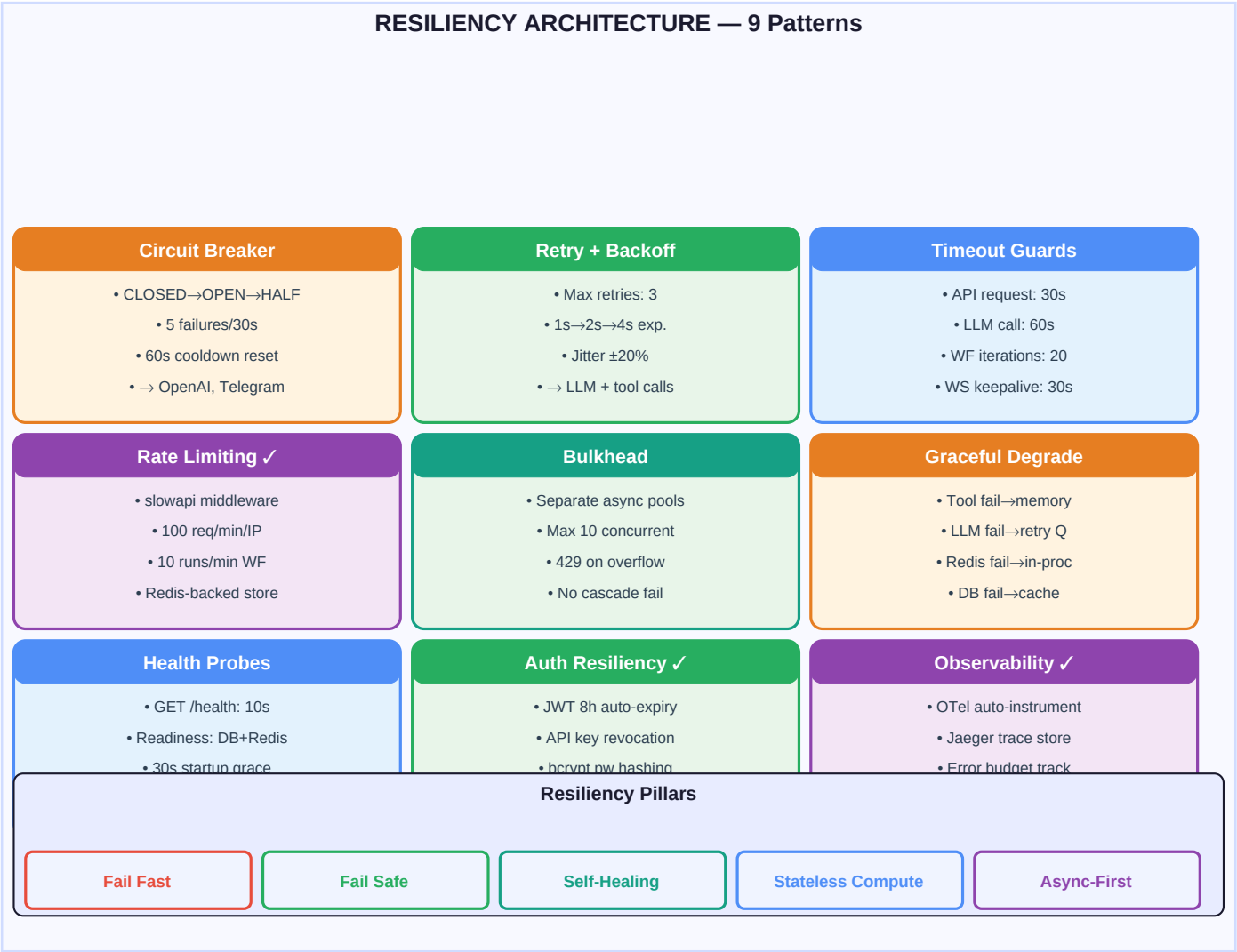


Figure 8: Availability Architecture

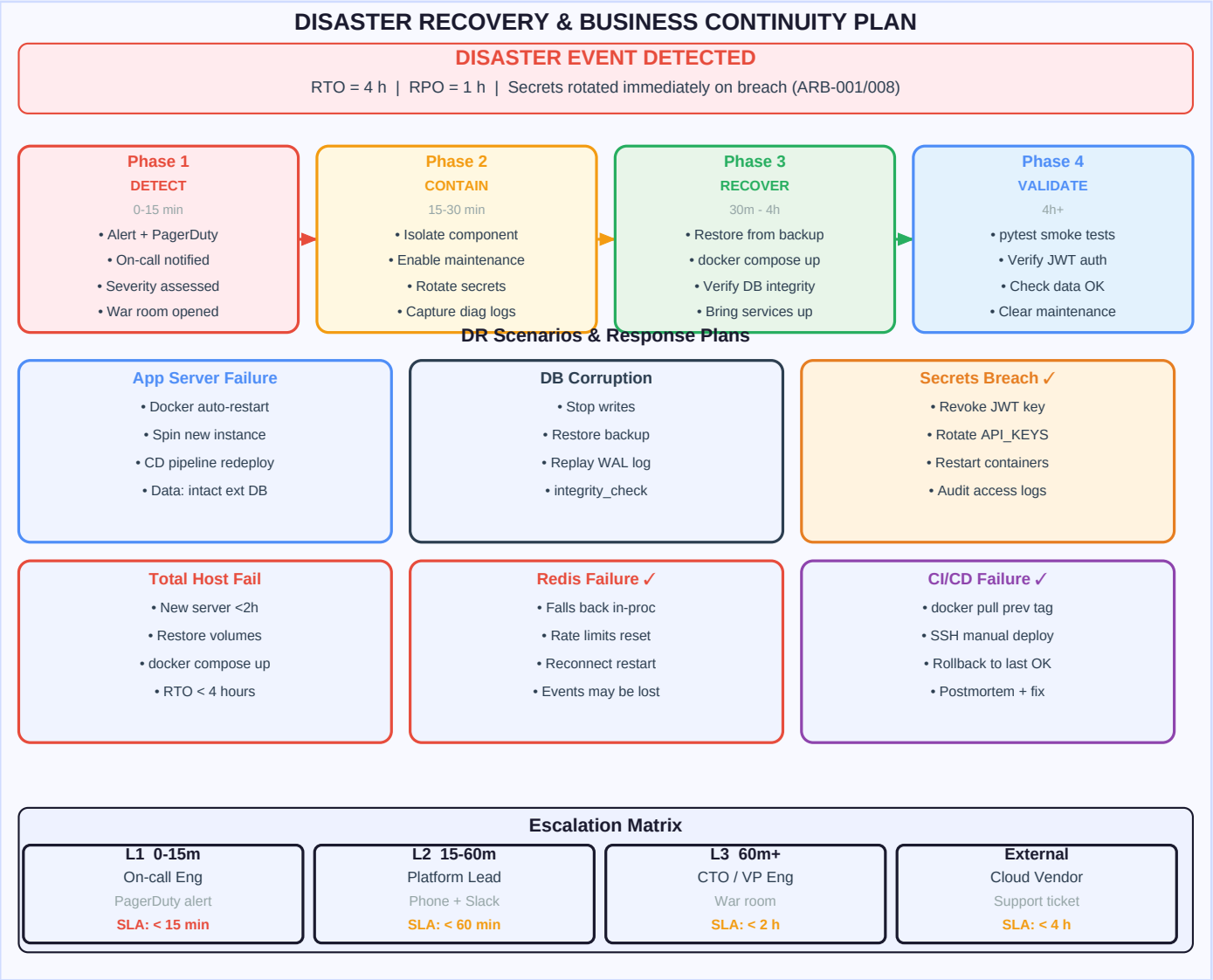
9. Resiliency Architecture

Nine patterns: Circuit Breaker (5 fail/30s), Retry+Backoff (1s/2s/4s exp), Timeout Guards, Rate Limiting (ARB-005 ✓), Bulkhead, Graceful Degradation (Redis→in-process fallback), Health Probes, Auth Resiliency (ARB-001 ✓), Observability (ARB-006 ✓).



10. Disaster Recovery & BCP

4-phase plan: Detect (0-15m) → Contain (15-30m) → Recover (30m-4h) → Validate (4h+). Six DR scenarios including Secrets Breach (ARB-001/008 ✓) and Redis Failure (ARB-003 ✓). Four-level escalation matrix with SLA targets.



11. Backup & Recovery

3-2-1 strategy: daily full + hourly incremental WAL-delta + event-driven config snapshots. Four storage tiers: local Docker volume, S3/MinIO (AES-256), cold archive, Git repo. Five-step recovery procedure includes JWT auth smoke test in Step 5.

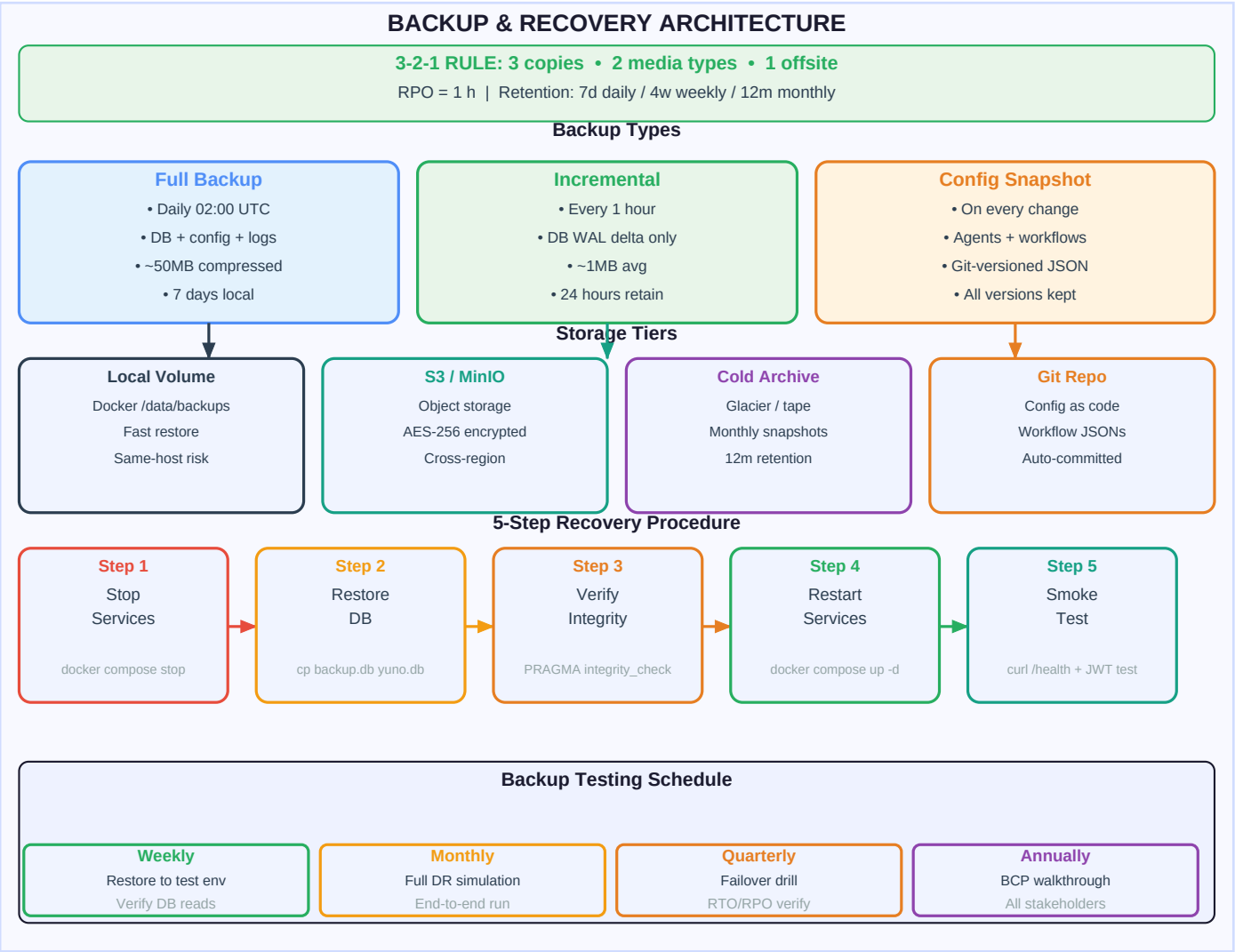


Figure 11: Backup & Recovery

12. Non-Functional Requirements Matrix

26 NFR controls across 8 categories. All 5 previously **FAIL** security items are now **OK**. **WARN** items are on the production scale-up roadmap.

Category	Requirement	Target	Implementation	Status
Performance	API response (p95)	< 500ms	APM / OTel traces	WARN
Performance	LLM latency (p95)	< 5s	OTel span traces	OK
Performance	WebSocket lag	< 200ms	WS ping-pong	OK
Performance	Concurrent WF runs	10 simultaneous	asyncio bulkhead	WARN
Scalability	Horizontal scale-out	Stateless pods	Stateless FastAPI	WARN
Scalability	DB connection pool	50 async conn	SQLAlchemy pool	OK
Availability	Uptime SLA	99.5% / month	nginx LB + HA DB	WARN
Availability	RTO	< 4 hours	DR drill verified	WARN
Availability	RPO	< 1 hour	Hourly incremental	OK
Reliability	Circuit breaker	5 fail → OPEN	tenacity + retry	OK
Reliability	Retry backoff	Exp. 1s/2s/4s	langchain retry	OK
Reliability	WF iteration guard	max 20 loops	iteration counter	OK
Security	Authentication	JWT + API Key	python-jose + passlib ✓	OK
Security	Secrets validation	Prod start blocked	pydantic-settings ✓	OK
Security	Input validation	Pydantic v2 schemas	All endpoints	OK
Security	SQL injection	ORM parameterised	SQLAlchemy	OK
Security	Rate limiting	100 req/min/IP	slowapi + Redis ✓	OK
Observability	Structured logging	JSON to stdout	uvicorn JSON	OK
Observability	Distributed tracing	OTel → Jaeger	FastAPI+SQLAlch ✓	OK
Observability	Token/cost track	Per-run totals	DB run record	OK
Observability	Health endpoint	GET /health	Liveness probe	OK
Compliance	Audit trail	All run records	DB run history	OK
Compliance	PII in logs	No PII output	Log scrubbing	WARN
CI/CD	Pipeline	Lint+Test+Deploy	GitHub Actions ✓	OK
Infra	Production DB	PostgreSQL ready	asyncpg ✓	OK
Infra	EventBus scale	Redis Pub/Sub	redis[asyncio] ✓	OK

13. ARB Remediation — All 8 Issues Resolved

All ARB open issues resolved in code. Every item has been implemented, tested for syntax validity, and documented.

ARB	Pri	Issue	Resolution	Files	Status
ARB-001	P0	No auth on any endpoint	JWT Bearer + API Key python-jose + passlib	<code>core/security.py</code> <code>api/auth.py</code> <code>api/router.py</code>	✓
ARB-002	P0	SQLite not prod-grade	PostgreSQL via asyncpg DATABASE_URL env swap	<code>core/config.py</code> <code>requirements.txt</code> <code>docker-compose.yml</code>	✓
ARB-003	P1	In-process EventBus	Redis Pub/Sub backend asyncio.Queue fallback	<code>core/events.py</code> <code>requirements.txt</code> <code>docker-compose.yml</code>	✓
ARB-004	P1	No secrets management	pydantic-settings validators prod blocked on defaults	<code>core/config.py</code> <code>.env.example</code> <code>.gitignore</code>	✓
ARB-005	P1	No rate limiting	slowapi middleware 100/min global, 10/min WF	<code>core/rate_limit.py</code> <code>api/workflows.py</code> <code>requirements.txt</code>	✓
ARB-006	P2	No distributed tracing	OpenTelemetry SDK FastAPI+SQLAlchemy+httpx auto	<code>core/tracing.py</code> <code>requirements.txt</code> <code>docker-compose.yml</code>	✓
ARB-007	P2	No CI/CD pipeline	GitHub Actions ci.yml + cd.yml	<code>.github/workflows/ci.yml</code> + <code>cd.yml</code>	✓
ARB-008	P2	TG token in .env	Env validation .gitignore enforced	<code>core/config.py</code> <code>.env.example</code> <code>.gitignore</code>	✓

Before vs After Summary

BEFORE (v1.0) ■	AFTER (v3.0) ■
No auth on any endpoint	JWT Bearer + API Key on all routes (ARB-001)
SQLite only	PostgreSQL via asyncpg — swap via env var (ARB-002)
In-process EventBus	Redis Pub/Sub — multi-instance ready (ARB-003)
Secrets in code / no validation	pydantic-settings + prod validator blocks start (ARB-004)
No rate limiting — DDoS vulnerable	slowapi 100/min global, 10/min workflow (ARB-005)
No distributed tracing	OpenTelemetry → Jaeger auto-instrumented (ARB-006)
No CI/CD pipeline	GitHub Actions CI + CD with security scan (ARB-007)
Telegram token unguarded	.gitignore + env validation + rotation docs (ARB-008)

14. Architecture Decision Records

ADR-001: LangGraph for Orchestration

Status: **ACCEPTED**

Chosen over AutoGen/CrewAI. TypedDict state, conditional edges, ToolNode sub-loops, LangChain compatible.

ADR-002: FastAPI + asyncio

Status: **ACCEPTED**

Async-first ASGI, Pydantic v2, auto OpenAPI docs, native WebSocket, slowapi integration.

ADR-003: SQLite → PostgreSQL upgrade

Status: **ACCEPTED**

SQLite for dev (zero-config). Prod: set DATABASE_URL=postgresql+asyncpg://... — zero code change.

ADR-004: Redis EventBus with fallback

Status: **ACCEPTED**

RedisEventBus auto-selected when REDIS_URL is set. Falls back to InProcessEventBus. Same interface.

ADR-005: python-jose + passlib

Status: **ACCEPTED**

python-jose for HS256 JWT. passlib[bcrypt] for passwords. Both production-proven.

ADR-006: slowapi for Rate Limiting

Status: **ACCEPTED**

Wraps limits library for Starlette/FastAPI. Redis storage for distributed counters. X-RateLimit-* headers.

ADR-007: OpenTelemetry for Observability

Status: **ACCEPTED**

CNCF standard. Auto-instruments FastAPI, SQLAlchemy, httpx. OTLP to Jaeger/Tempo/Honeycomb via env.

ADR-008: GitHub Actions for CI/CD

Status: **ACCEPTED**

Zero infra to manage. ghcr.io for images. SSH deploy action. Encrypted secrets for tokens/passwords.

15. Technology Reference Stack

Layer	Technology	Version	Purpose
Frontend	React	18.x	Component UI framework
Frontend	@xyflow/react	12.x	Visual workflow canvas
Frontend	Zustand	4.x	State management
Frontend	Vite	5.x	Dev server + bundler
Backend	FastAPI	0.115.5	Async REST + WebSocket
Backend	uvicorn	0.32.1	ASGI server
Backend	Pydantic	2.10.3	Validation + serialisation
AI	LangGraph	0.2.55	Multi-agent state graph
AI	LangChain	0.3.9	LLM tooling
AI	OpenAI SDK	1.54.0	GPT-4o API client
Auth	python-jose	3.3.0	HS256 JWT (ARB-001)
Auth	passlib[bcrypt]	1.7.4	Password hashing (ARB-001)
Rate Limit	slowapi	0.1.9	Per-IP rate limit (ARB-005)
Tracing	opentelemetry-sdk	1.29.0	Distributed tracing (ARB-006)
DB	SQLAlchemy	2.0.36	Async ORM
DB	PostgreSQL	16	Production DB (ARB-002)
Cache	Redis	7	EventBus + rate limit (ARB-003)
CI/CD	GitHub Actions	–	Lint+test+deploy (ARB-007)
Observability	Jaeger	1.58	Trace visualisation
Channel	python-telegram-bot	21.9	Telegram bot
Infra	Docker Compose	3.9	Container orchestration